

Tài liệu Đặc tả Thiết kế Giải pháp (Solution Design Specification - SDS)

QuickAssist – Google Extension Trợ lý Ghi chú nhanh dựa trên AI
Lê Đăng Sơn, Mai Đức Vinh, Trịnh Mạnh Quang, Trần Thùy Dương
Phiên bản 1.1 – 02/10/2026

Mục lục

1. Giới thiệu.....	4
1.1. Mục đích.....	4
1.2. Tổng quan dự án.....	4
1.3. Phạm vi.....	4
1.4. Tài liệu tham khảo.....	5
2. Giả định và ràng buộc.....	5
2.1. Giả định.....	5
2.2. Ràng buộc.....	6
3. Yêu cầu chức năng và yêu cầu phi chức năng.....	6
3.1. Yêu cầu chức năng.....	6
3.2. Yêu cầu phi chức năng.....	7
4. Kiến trúc Giải pháp.....	8
4.1. Sơ đồ kiến trúc logic.....	8
4.2. Sơ đồ kiến trúc vật lý.....	9
4.3. Sơ đồ kiến trúc các thành phần.....	9
4.4. Quyết định kiến trúc và đánh đổi.....	10
4.5. Ước lượng tải và điểm nghẽn.....	11
4.6. Tổ chức mã nguồn theo tầng.....	12
5. Thiết kế chi tiết.....	12
5.1. Sequence Diagram.....	12
5.1.1. Sequence Diagram đăng nhập bằng Google OAuth.....	13
5.1.2. Sequence Diagram lưu ghi chú và tạo embedding.....	14
5.1.3. Sequence Diagram tìm kiếm ngữ nghĩa.....	15
5.1.4. Sequence Diagram tóm tắt nội dung bằng AI.....	16
5.1.5. Sequence Diagram xóa tài khoản và dữ liệu.....	17
5.1.6. Sequence Diagram hỏi đáp AI dựa trên ghi chú cá nhân.....	18
5.2. Các dịch vụ.....	18
5.3. Danh mục API.....	19
6. Thiết kế dữ liệu.....	20
7. Công nghệ sử dụng.....	21
8. Bảo mật.....	21
8.1. Biện pháp bảo mật.....	21
9. Triển khai.....	22
10. Xử lý lỗi và Giảm thiểu rủi ro.....	23

Quản lý tài liệu

Phiên bản	1.1
Trạng thái	Dự thảo (Draft)
Ngày	02/10/2026
Tác giả	Lê Đăng Sơn, Mai Đức Vinh, Trịnh Mạnh Quang, Trần Thùy Dương
Giảng viên hướng dẫn	TS. Nguyễn Trọng Phúc

Bảng 1. Lịch sử phiên bản

Phiên bản	Ngày	Tác giả	Nội dung thay đổi
1.0	04/09/2026	L.D. Sơn, M.D. Vinh, T.M. Quang, T.T. Dương	Viết lại theo mẫu SDS; xác thực dùng Google OAuth; loại bỏ lịch trình cá nhân.
1.1	02/10/2026	L.D. Sơn, M.D. Vinh, T.M. Quang, T.T. Dương	Thống nhất kiến trúc phân tầng (bỏ mô tả microservice ở §5.2); vẽ lại Hình 1–9 khớp thiết kế triển khai; bổ sung Quota Service, quyết định kiến trúc và đánh đổi (§4.4), ước lượng tải (§4.5), tổ chức mã nguồn (§4.6), danh mục API (§5.3); viết lại thiết kế dữ liệu (§6), chốt công nghệ (§7); cập nhật bảo mật, triển khai và rủi ro.

1. Giới thiệu

1.1. Mục đích

Tài liệu Đặc tả Thiết kế Giải pháp này mô tả thiết kế tổng thể cho dự án QuickAssist, gồm phạm vi, yêu cầu, kiến trúc, dữ liệu, công nghệ, bảo mật, triển khai và phương án xử lý rủi ro. Tài liệu thống nhất cách hiểu giữa các thành viên trước khi phát triển, là căn cứ để phân công, xây dựng, kiểm thử và nghiệm thu sản phẩm.

Đối tượng đọc tài liệu gồm nhóm phát triển, nhóm kiểm thử, người phụ trách triển khai và giảng viên hướng dẫn. Các quyết định ở mức giải pháp được trình bày trong SDS kèm lý do lựa chọn và đánh đổi (trade-off); thuật toán và mã nguồn cụ thể được mô tả trong kho mã nguồn (thư mục docs/architecture và docs/adr).

1.2. Tổng quan dự án

Trong quá trình học tập và làm việc, người dùng thường phải đọc nhiều tài liệu trên website. Cách ghi chú hiện nay yêu cầu sao chép nội dung, chuyển sang ứng dụng khác, dán, lưu đường dẫn và tự phân loại. Khi số lượng ghi chú tăng, việc tìm lại trở nên khó khăn nếu người dùng không nhớ đúng tiêu đề hoặc từ khóa ban đầu.

QuickAssist là tiện ích mở rộng trên trình duyệt giúp người dùng lưu nhanh đoạn văn bản đang đọc, tổ chức thành thư viện cá nhân, tóm tắt bằng AI và tra cứu theo ý nghĩa. Đối tượng chính của phiên bản đầu là sinh viên và người học thường xuyên đọc tài liệu web.

Người dùng bôi đen nội dung rồi chọn lệnh từ menu chuột phải hoặc sử dụng phím tắt. Hệ thống lưu nội dung cùng tiêu đề trang, đường dẫn và thời gian. Khi truy vấn, hệ thống có thể trả trực tiếp các đoạn ghi chú liên quan hoặc, nếu người dùng lựa chọn, sử dụng mô hình ngôn ngữ để tạo câu trả lời có nguồn.

Mô hình triển khai chính là dịch vụ trực tuyến do nhóm vận hành. Việc công khai mã nguồn để người dùng tự triển khai là định hướng tương lai và không thuộc phạm vi nghiệm thu của phiên bản môn học.

1.3. Phạm vi

Trong phạm vi (In-scope):

- **Tính năng cốt lõi (Must-have):** Đăng nhập Google; lưu, quản lý và phân loại ghi chú (thư mục); tóm tắt bằng AI; tạo embedding và tìm kiếm ngữ nghĩa; quản lý quota AI; bảo vệ và xóa dữ liệu; ghi log hoạt động và lỗi.
- **Tính năng mở rộng (Nice-to-have):** Chatbot AI đa lượt với dữ liệu cá nhân (streaming, nhớ 5 lượt thoại); tiện ích giao diện (phím tắt, gợi ý thư mục, gắn thẻ/lọc, xuất dữ liệu); hỗ trợ thêm trình duyệt Chromium.

Ngoài phạm vi (Out-of-scope):

- **Nền tảng và vận hành:** Không phát hành app Mobile/Desktop; không hỗ trợ Firefox/Safari cho MVP; không cam kết vận hành thương mại/production; không cấp bộ cài self-host.

- **Thu thập và xử lý dữ liệu:** Không thu thập ngầm lịch sử duyệt web; không xử lý file đa phương tiện (ảnh, video, PDF, bảng phức tạp); không trích xuất sự kiện/đồng bộ lịch; không có chức năng tìm đoạn nổi bật trên trang (skim).
- **AI và cộng tác:** Chatbot không dùng kiến thức tổng quát ngoài ghi chú; không huấn luyện (fine-tune) mô hình riêng; không hỗ trợ chia sẻ hay làm việc nhóm.

1.4. Tài liệu tham khảo

STT	Tài liệu
[1]	Báo cáo kế hoạch thực hiện môn học – QuickAssist, Nhóm 1.
[2]	Kịch bản chức năng QuickAssist; Kế hoạch phân công QuickAssist v2.
[3]	SDS mẫu: Tự động hóa xử lý email yêu cầu hỗ trợ khách hàng bằng AI.
[4]	DesignGurus – System Design Trade-offs: A Step-by-Step Framework (tài liệu tham khảo khung đánh đổi thiết kế).
[5]	Chrome Extensions Documentation, https://developer.chrome.com/docs/extensions/
[6]	Google Identity – OAuth 2.0 / OpenID Connect, https://developers.google.com/identity/protocols/oauth2
[7]	OpenAI API Documentation, https://platform.openai.com/docs/
[8]	FastAPI Documentation, https://fastapi.tiangolo.com/
[9]	PostgreSQL Documentation, https://www.postgresql.org/docs/ ; pgvector, https://github.com/pgvector/pgvector
[10]	Kho mã nguồn quick-assist-ext: docs/architecture (kiến trúc), docs/adr (quyết định kiến trúc), docs/api (quy ước API).

2. Giả định và ràng buộc

2.1. Giả định

- **Người dùng và môi trường:** Người dùng có tài khoản Google, kết nối Internet ổn định cho các tính năng AI/đồng bộ, và trang web cho phép đọc văn bản được chọn.
- **Hệ thống và cấu hình:** Backend, cơ sở dữ liệu và các API đã được thiết lập sẵn sàng. Dữ liệu được cô lập theo từng tài khoản. Các tham số AI (model, kích thước chunk, top-k, hạn mức quota) cấu hình được ngoài mã nguồn qua biến môi trường.
- **Kiểm thử:** Dữ liệu dùng để kiểm thử là dữ liệu an toàn, không chứa thông tin nhạy cảm thật. Môi trường phát triển và CI dùng AI giả lập (mock) để không phụ thuộc mạng và chi phí.

2.2. Ràng buộc

- **Dự án và nguồn lực:** Nhóm 4 người trong 8 tuần của học phần; MVP định hướng cloud-first (không bắt buộc bàn giao bản self-host). Kiến trúc phải đủ đơn giản để vận hành bởi nhóm nhỏ nhưng có ranh giới rõ để phân công song song.
- **Công nghệ và kỹ thuật:** Extension tuân thủ Chrome Manifest V3; giao tiếp client-server qua HTTPS/JSON; bắt buộc dùng PostgreSQL kèm pgvector để lưu trữ dữ liệu và vector.
- **Phụ thuộc bên thứ ba:** Hệ thống bị ảnh hưởng bởi quota, chi phí và độ ổn định của Google OAuth và OpenAI API.
- **Giới hạn tính năng và bảo mật:** Chỉ hỗ trợ văn bản (có thể lỗi với web động, paywall, web chống sao chép). Nội dung thu thập được xử lý như dữ liệu không tin cậy để đề phòng mã độc và prompt injection từ website đánh lừa AI.

3. Yêu cầu chức năng và yêu cầu phi chức năng

Phần này mô tả các yêu cầu chức năng và phi chức năng mà hệ thống QuickAssist cần đáp ứng.

3.1. Yêu cầu chức năng

Các yêu cầu này xác định những chức năng cụ thể mà hệ thống phải thực hiện.

Chức năng bắt buộc (Must-have)

- Hệ thống phải cho phép người dùng đăng nhập và đăng xuất bằng tài khoản Google thông qua Google OAuth.
- Hệ thống phải cho phép người dùng bôi đen một đoạn văn bản trên trang web và lưu nội dung đó vào QuickAssist thông qua menu chuột phải (và phím tắt Ctrl+Alt+N), có thể hoàn tác trong 5 giây.
- Khi tạo ghi chú, hệ thống phải thu thập và lưu các thông tin cơ bản gồm nội dung được chọn, tiêu đề trang, đường dẫn, tên miền và thời gian tạo.
- Hệ thống phải cho phép người dùng xem, chỉnh sửa và xóa các ghi chú thuộc tài khoản của mình.
- Hệ thống phải cho phép người dùng tạo thư mục và sắp xếp các ghi chú vào từng thư mục.
- Nếu người dùng không chọn thư mục khi tạo ghi chú, hệ thống phải lưu ghi chú vào thư mục mặc định Inbox.
- Hệ thống phải chia nội dung ghi chú thành các đoạn nhỏ và tạo vector embedding để phục vụ tìm kiếm theo ngữ nghĩa.
- Hệ thống phải cho phép người dùng nhập nội dung cần tìm kiếm và trả về các đoạn ghi chú có ý nghĩa liên quan nhất.
- Kết quả tìm kiếm phải hiển thị nội dung liên quan cùng tiêu đề ghi chú và đường dẫn của trang web nguồn.
- Hệ thống phải cho phép người dùng yêu cầu tóm tắt ghi chú hoặc đoạn văn bản được chọn thông qua OpenAI API và chỉ lưu bản tóm tắt khi người dùng xác nhận.

- Hệ thống phải kiểm tra quota của tài khoản trước khi thực hiện các chức năng sử dụng OpenAI API.
- Hệ thống phải ghi nhận lượng token đã sử dụng để quản lý quota của từng tài khoản.
- Hệ thống phải cho phép người dùng yêu cầu xóa tài khoản và toàn bộ dữ liệu ghi chú có liên quan.
- Hệ thống phải ghi log các hoạt động xử lý chính và các lỗi xảy ra trong quá trình hoạt động.

Chức năng mở rộng (Nice-to-have): Các chức năng dưới đây được triển khai khi nhóm còn đủ thời gian và không phải điều kiện bắt buộc để nghiệm thu phiên bản MVP.

- Hệ thống có thể đề xuất thư mục phù hợp dựa trên tiêu đề, đường dẫn, tên miền và nội dung của trang web.
- Hệ thống có thể cho phép người dùng gắn thẻ, lọc và sắp xếp các ghi chú.
- Hệ thống có thể cung cấp chức năng hỏi đáp bằng AI dựa trên những ghi chú được tìm thấy trong thư viện của người dùng.
- Câu trả lời do AI tạo có thể được hiển thị theo dạng streaming và kèm theo nguồn ghi chú đã sử dụng.
- Hệ thống có thể hỗ trợ hội thoại nhiều lượt và sử dụng tối đa năm cặp hỏi–đáp gần nhất để duy trì ngữ cảnh của phiên trò chuyện.
- Hệ thống có thể cho phép người dùng dừng quá trình tạo câu trả lời đang diễn ra.

3.2. Yêu cầu phi chức năng

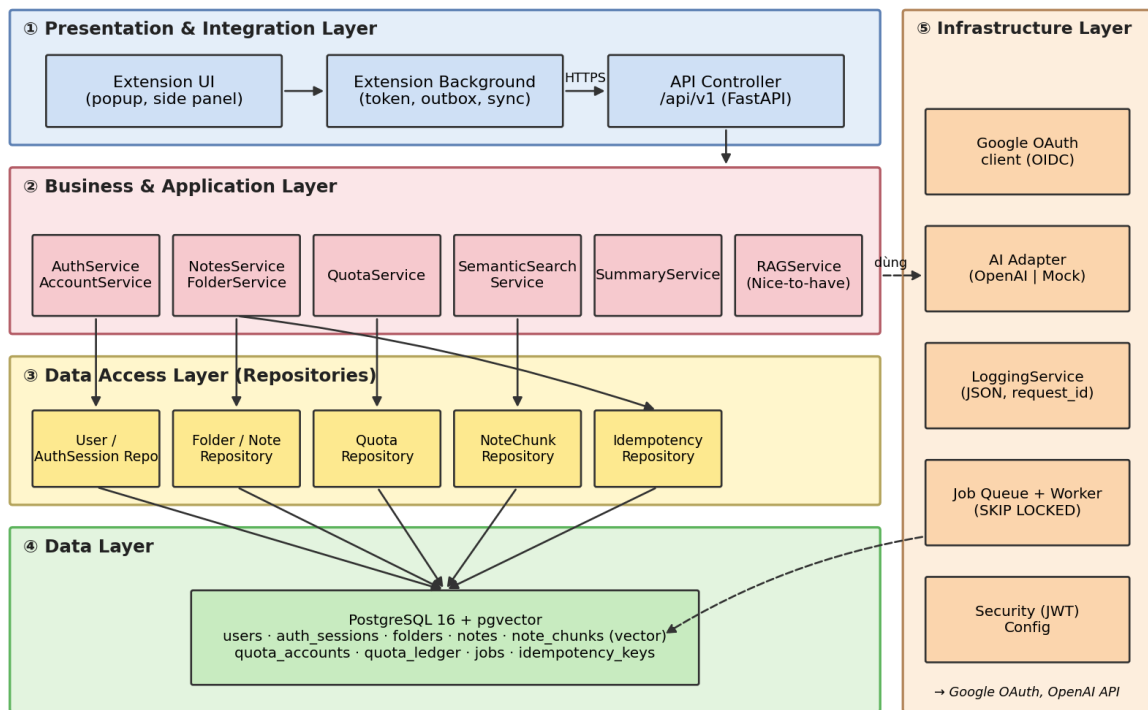
Các yêu cầu này xác định những tiêu chuẩn về chất lượng và cách thức hoạt động của hệ thống.

- **Hiệu năng:** Lưu ghi chú dưới 2 giây (không chờ AI xử lý); tìm kiếm dưới 3 giây trong điều kiện thông thường; mở thư viện từ bộ nhớ đệm dưới 200 ms.
- **Bảo mật:** Sử dụng HTTPS, Google OAuth (Authorization Code + PKCE); bảo đảm người dùng chỉ truy cập dữ liệu của mình.
- **Riêng tư:** Chỉ thu thập nội dung khi người dùng chủ động lưu hoặc yêu cầu tóm tắt; không lưu khóa API, token hay thông tin nhạy cảm trong mã nguồn/log.
- **Độ tin cậy:** Ghi chú gốc vẫn được lưu nếu tạo embedding lỗi và có thể xử lý lại; tránh tạo ghi chú trùng khi lưu liên tiếp hoặc gửi lại do mất mạng.
- **Khả năng mở rộng và bảo trì:** API không lưu trạng thái phiên (stateless) để chạy nhiều instance; xử lý AI chạy ở worker riêng; các tham số AI, quota và tìm kiếm cấu hình linh hoạt; mã nguồn tổ chức theo tầng có kiểm tra tự động.
- **Khả năng sử dụng:** Giao diện thông báo rõ trạng thái xử lý, thành công, lỗi, hết quota hoặc không có kết quả.
- **Giới hạn tài nguyên:** Tuân thủ quota, token và giới hạn yêu cầu của OpenAI API; giới hạn độ dài ghi chú và văn bản tóm tắt.
- **Lưu trữ dữ liệu:** Dữ liệu được lưu tối đa một năm theo chính sách tài khoản; cơ chế thực hiện là tác vụ nền định kỳ dọn dữ liệu của tài khoản không hoạt động quá một năm (dựa trên last_active_at).

4. Kiến trúc Giải pháp

QuickAssist áp dụng **kiến trúc phân tầng (Layered Architecture)**. Phía máy chủ được triển khai dạng **monolith có phân module**: một codebase, hai tiến trình (API và Worker), dùng chung một cơ sở dữ liệu PostgreSQL + pgvector. Các “service” trong tài liệu này là các thành phần của Business Layer, gọi nhau qua giao diện công khai trong cùng tiến trình, không phải microservice triển khai riêng (lý do ở §4.4).

4.1. Sơ đồ kiến trúc logic

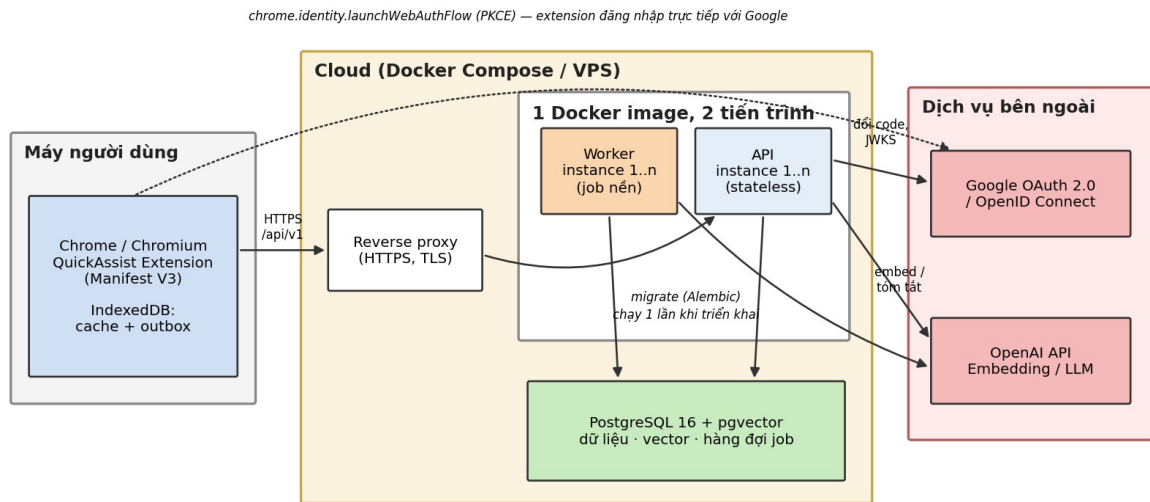


Hình 1. Sơ đồ kiến trúc logic (phân tầng)

- **Presentation & Integration Layer:** Extension UI (popup, side panel) chỉ giao tiếp với Extension Background; Background là cổng duy nhất gọi API Controller qua HTTPS, quản lý token, hàng đợi gửi lại (outbox) và đồng bộ. API Controller nhận/kiểm tra dữ liệu, gọi service và đổi lỗi nghiệp vụ thành phản hồi HTTP; không chứa nghiệp vụ, không truy cập CSDL.
- **Business & Application Layer:** AuthService/AccountService, NotesService/FolderService, QuotaService, SemanticSearchService, SummaryService, RAGService. Đây là nơi chứa nghiệp vụ, quyết định transaction (commit/rollback), kiểm tra quota và phát tác vụ nền.
- **Data Access Layer:** Mỗi nhóm bảng có một repository riêng (User/AuthSession, Folder/Note, Quota, NoteChunk, Idempotency). Repository chỉ chứa truy vấn; mọi truy vấn dữ liệu người dùng đều lọc theo user_id.
- **Data Layer:** PostgreSQL 16 với pgvector lưu dữ liệu quan hệ, vector embedding, chỉ mục full-text và hàng đợi tác vụ.
- **Infrastructure Layer:** Client Google OAuth (OIDC), AI Adapter (OpenAI hoặc Mock), LoggingService, hàng đợi tác vụ và Worker, tiện ích bảo mật (JWT) và cấu hình.

Quy tắc phụ thuộc giữa các tầng (được kiểm tra tự động trong CI): (1) tầng trên chỉ gọi tầng dưới, không gọi ngược; (2) Presentation không truy cập Data Access/Data; (3) service chỉ gọi service khác qua giao diện công khai; (4) service chỉ dùng repository của bảng mà nó sở hữu; (5) Business không viết SQL; (6) repository không commit transaction; (7) Infrastructure không phụ thuộc nghiệp vụ.

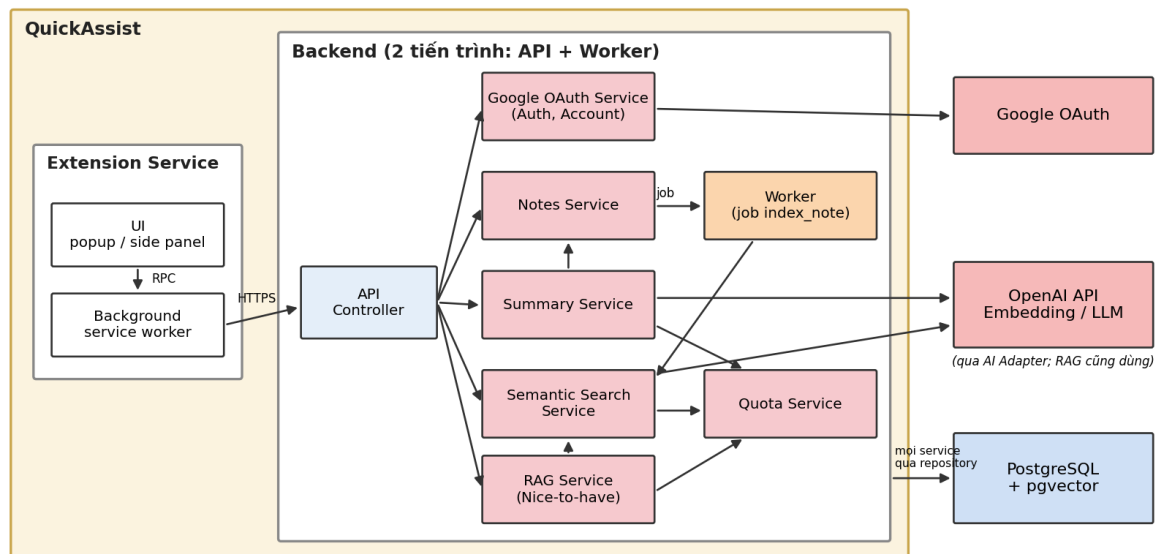
4.2. Sơ đồ kiến trúc vật lý



Hình 2. Sơ đồ kiến trúc vật lý

MVP sử dụng kiến trúc cloud-first. Extension chạy trên trình duyệt của người dùng và lưu bộ nhớ đệm, hàng đợi gửi lại trong IndexedDB. Phía cloud chạy bằng Docker Compose: reverse proxy kết thúc HTTPS; cùng một Docker image chạy hai tiến trình là API (FastAPI, không lưu trạng thái phiên) và Worker (xử lý tác vụ nền); PostgreSQL + pgvector là dịch vụ dữ liệu riêng. Google OAuth và OpenAI API là hệ thống bên ngoài. API và Worker có thể chạy nhiều instance mà không cần thay đổi mã nguồn.

4.3. Sơ đồ kiến trúc các thành phần



Hình 3. Sơ đồ kiến trúc các thành phần

- **Extension Service:** Giao diện chính của QuickAssist trên trình duyệt. UI gửi yêu cầu (RPC) tới Background; Background thực hiện đăng nhập, lưu ghi chú qua outbox, đồng bộ thư viện và gọi API.
- **API Controller:** Điểm vào duy nhất của backend (/api/v1): xác thực access token, kiểm tra dữ liệu đầu vào, gọi service tương ứng, trả lỗi theo định dạng thống nhất.
- **Google OAuth Service (AuthService, AccountService):** Đổi authorization code, xác minh danh tính Google, tạo hoặc khôi phục tài khoản, phát và thu hồi app session; quản lý hồ sơ và xóa tài khoản. Không lưu mật khẩu Google.
- **Notes Service:** Tạo, xem, sửa, xóa, phân loại ghi chú theo thư mục; chống tạo trùng bằng idempotency key; quản lý trạng thái xử lý AI; phát tác vụ đánh chỉ mục khi ghi chú thay đổi.
- **Quota Service:** Đánh giá hạn mức trước mỗi lệnh gọi OpenAI (giữ chỗ), ghi nhận chính xác token tiêu thụ theo request_id (chốt hoặc hoàn lại).
- **Semantic Search Service:** Chia đoạn, tạo embedding và lưu chỉ mục (thông qua Worker); tạo vector cho truy vấn và tìm kiếm kết hợp vector + từ khóa trong pgvector.
- **Summary Service:** Tạo bản tóm tắt xem trước bằng LLM; việc lưu do Notes Service thực hiện sau khi người dùng xác nhận.
- **RAG Service (Nice-to-have):** Ghép ngữ cảnh từ ghi chú và lịch sử hội thoại, điều phối streaming và dừng câu trả lời.
- **Worker:** Tiến trình nền lấy tác vụ từ hàng đợi trên PostgreSQL và thực thi (hiện tại: đánh chỉ mục ghi chú).
- **PostgreSQL + pgvector, Google OAuth, OpenAI API:** Lưu trữ dữ liệu và vector; dịch vụ xác thực bên ngoài; dịch vụ embedding/LLM bên ngoài (truy cập qua AI Adapter).

4.4. Quyết định kiến trúc và đánh đổi

Theo khung đánh đổi thiết kế [4], mỗi quyết định dưới đây đều được so sánh với ít nhất một phương án khác và nêu rõ cái giá chấp nhận. Thứ tự ưu tiên chất lượng đã thống nhất: (1) bảo mật và riêng tư, (2) không mất ghi chú, (3) dễ phân công cho nhóm 4 người, (4) độ trễ các thao tác chính, (5) khả năng mở rộng.

Bảng 2. Quyết định kiến trúc và đánh đổi

Quyết định	Phương án đã loại	Lý do chọn	Đánh đổi chấp nhận
Kiến trúc phân tầng, triển khai monolith 2 tiến trình (API + Worker)	Microservice cho từng service	Nhóm nhỏ, tải thấp; xóa tài khoản cần 1 transaction ACID; một lần triển khai	Không mở rộng riêng từng service; bù lại API/Worker vẫn mở rộng ngang được
PostgreSQL + pgvector là kho dữ liệu duy nhất	Vector DB riêng (Qdrant, Pinecone); Redis	Một nguồn sự thật; xóa dữ liệu nguyên tử bằng CASCADE; ít hạ tầng	Đổi model embedding khác số chiều cần migration + đánh chỉ mục lại
Hàng đợi tác vụ trên	Xử lý đồng bộ	Lưu ghi chú < 2 s; tác	Trạng thái READY

Quyết định	Phương án đã loại	Lý do chọn	Đánh đổi chấp nhận
PostgreSQL + outbox cùng transaction	trong request; Celery + Redis	vụ không mất khi khởi động lại; không có ghi chú “mồ côi”	trễ vài giây (nhất quán cuối cùng)
AI qua AI Adapter (OpenAI Mock)	Gọi SDK OpenAI trực tiếp trong service	Kiểm thử offline, đối nhà cung cấp bằng cấu hình	Không dùng được tính năng riêng của SDK nếu chưa mở rộng adapter
OAuth Code + PKCE, server đổi code, app session riêng (access 15 phút + refresh xoay vòng)	Dùng token Google trực tiếp làm phiên	Extension không giữ secret; API stateless; thu hồi được phiên, phát hiện tái sử dụng refresh token	Access token còn hiệu lực tối đa 15 phút sau khi thu hồi
Tìm kiếm kết hợp vector + full-text, gộp bằng Reciprocal Rank Fusion	Chỉ tìm theo vector	Tốt hơn với tên riêng/thuật ngữ; 1 truy vấn SQL; luôn lọc theo user_id	Full-text cấu hình simple chưa xử lý dấu tiếng Việt (đo và cải thiện sau)
Extension: Background là cổng mạng duy nhất; không có content script thường trực	UI tự gọi API; content script trên mọi trang	Một nơi quản lý token/retry; quyền tối thiểu (activeTab)	Thêm khai báo RPC cho mỗi tính năng
Cache-first khi đọc, outbox khi tạo ghi chú; máy chủ là nguồn sự thật	Offline-first đồng bộ hai chiều	Không mất ghi chú khi mất mạng; xử lý xung đột đơn giản (version, 409)	Không sửa ghi chú khi offline
SSE cho streaming câu trả lời (Nice-to-have)	WebSocket, long-polling	Chạy trên HTTP thường, đủ cho luồng một chiều	Mỗi câu hỏi là một request mới

Mức nhất quán dữ liệu: nhất quán mạnh (transaction ACID) cho ghi chú, thư mục, quota và xóa tài khoản; nhất quán cuối cùng (eventual consistency) cho chỉ mục tìm kiếm (ghi chú mới chuyển READY sau vài giây) và bộ nhớ đệm IndexedDB trên extension.

4.5. Ước lượng tải và điểm nghẽn

- **Ước lượng:** quy mô demo khoảng 50 người dùng $\times$ 30 ghi chú/ngày $\approx$ 1.500 ghi chú/ngày $\approx$ 0,02 tác vụ/giây; tìm kiếm lúc cao điểm dưới 5 yêu cầu/giây. Một instance API, một Worker và một PostgreSQL là đủ.
- **Điểm nghẽn đầu tiên:** giới hạn tốc độ (rate limit) và chi phí của OpenAI API, không phải máy chủ. Giảm thiểu bằng quota theo token, timeout, retry có giới hạn và AI Mock cho phát triển/kiểm thử.

- **Khi tải tăng:** chạy thêm instance API (stateless) và Worker (lấy tác vụ bằng FOR UPDATE SKIP LOCKED); thêm connection pool, read replica cho tìm kiếm; khi hàng đợi vượt vài trăm tác vụ/giây mới chuyển sang message broker.

4.6. Tổ chức mã nguồn theo tầng

Cấu trúc thư mục của mã nguồn ánh xạ 1–1 với kiến trúc ở §4.1, giúp đối chiếu SDS với mã nguồn và phân công theo thư mục.

Bảng 3. Đối chiếu thành phần SDS và thư mục mã nguồn (gốc: apps/api/src/quickassist)

Thành phần trong SDS	Thư mục mã nguồn
Extension UI / Background	apps/extension/src/ui, apps/extension/src/background (+ content, shared)
API Controller (Presentation)	apps/api/src/quickassist/presentation/api/v1/controllers, presentation/schemas
AuthService, AccountService	business/auth
NotesService, FolderService	business/notes
QuotaService	business/quota
SemanticSearchService (+ chunking, job đánh chỉ mục)	business/semantic_search
SummaryService	business/summary
RAGService	business/rag
Data Access Layer (Repositories)	data_access/repositories
Data Layer (entity, kết nối, migration)	data/models, data/database.py, apps/api/migrations
Google OAuth, OpenAI API, LoggingService, Job Queue	infrastructure/google_oauth, infrastructure/openai, infrastructure/logging.py, infrastructure/jobs
Tiến trình API / Worker	main.py, worker.py

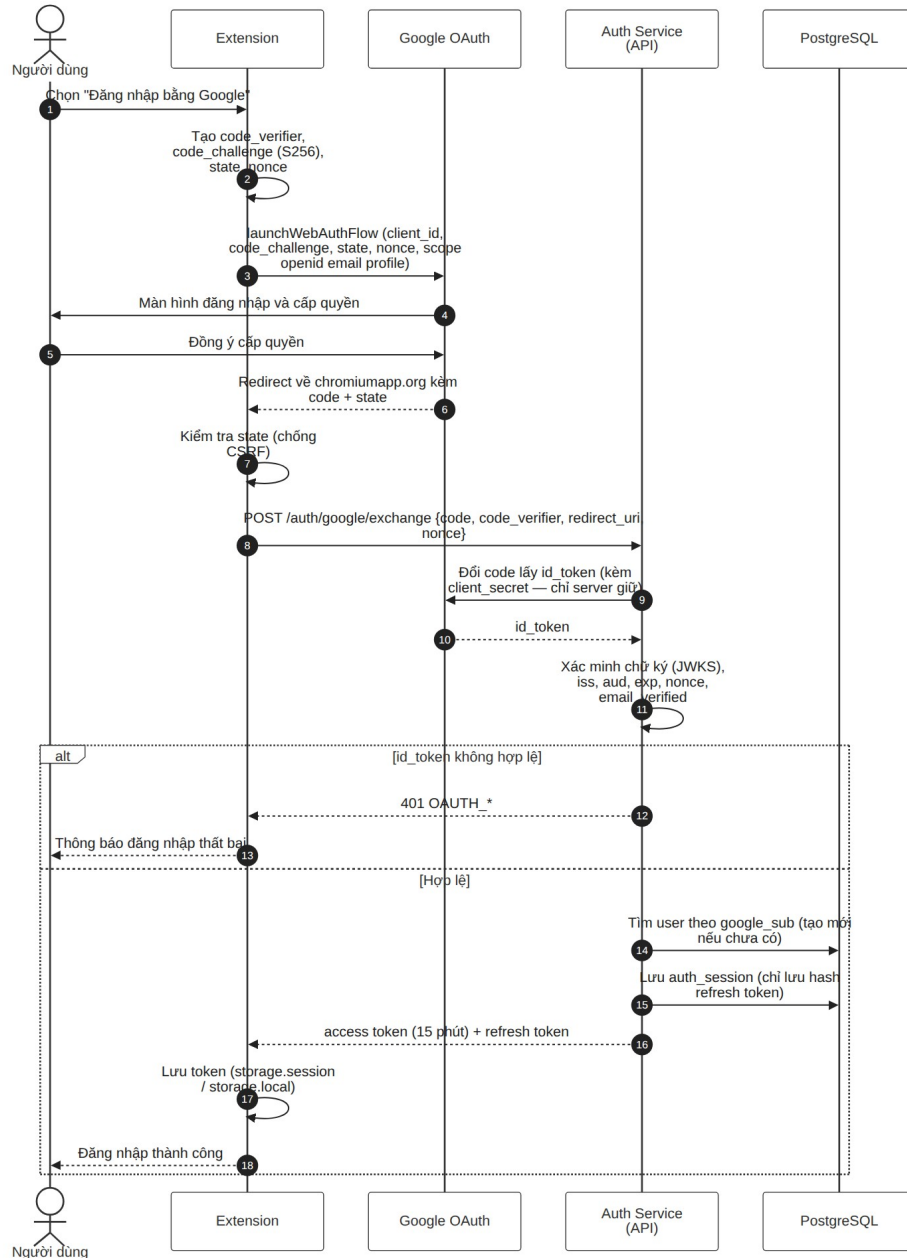
5. Thiết kế chi tiết

Phần này mô tả các luồng nghiệp vụ cốt lõi và cấu trúc dịch vụ của hệ thống QuickAssist. Các thiết kế được tinh chỉnh để quản lý tài nguyên chặt chẽ, tối ưu hiệu suất thông qua xử lý bất đồng bộ và đảm bảo tính toàn vẹn dữ liệu.

5.1. Sequence Diagram

Các biểu đồ tuần tự mô tả chi tiết quá trình tương tác giữa Extension, các service của backend và hệ thống bên ngoài. Tên service trên sơ đồ là thành phần của Business Layer; mọi yêu cầu từ Extension đi qua API Controller (lược bỏ trên sơ đồ cho gọn).

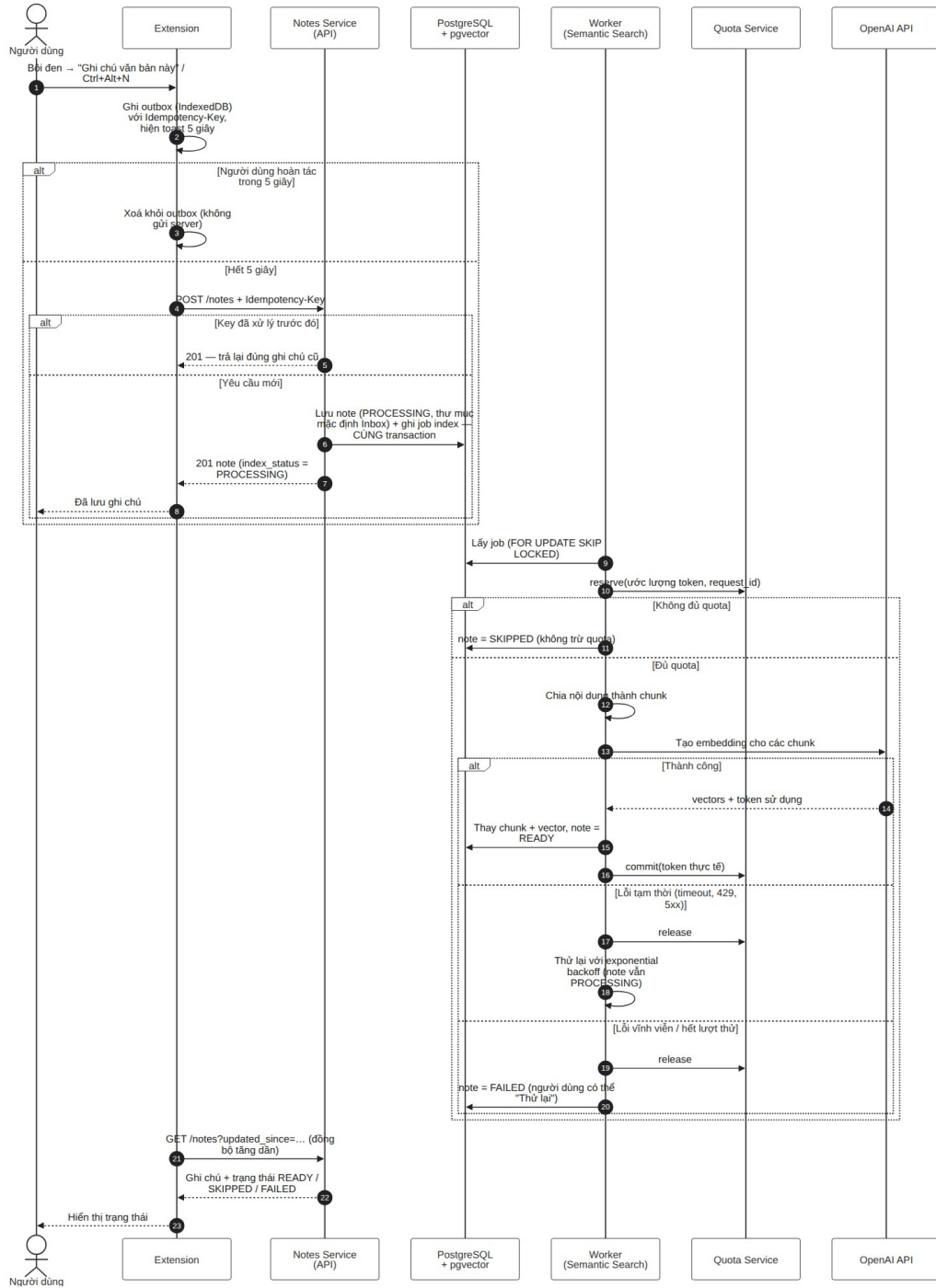
5.1.1. Sequence Diagram đăng nhập bằng Google OAuth



Hình 4. Sequence Diagram đăng nhập bằng Google OAuth

Extension tự khởi chạy luồng Authorization Code + PKCE với Google (chrome.identity.launchWebAuthFlow) và kiểm tra state khi nhận code. Code, code_verifier và nonce được gửi lên Auth Service; server dùng client_secret (chỉ server giữ) để đổi lấy id_token, xác minh chữ ký và các trường iss, aud, exp, nonce, email_verified. Tài khoản được tạo mới nếu chưa tồn tại (thư mục Inbox được tạo ở lần sử dụng đầu tiên). Server phát app session gồm access token 15 phút và refresh token (chỉ lưu giá trị băm), xoay vòng mỗi lần làm mới.

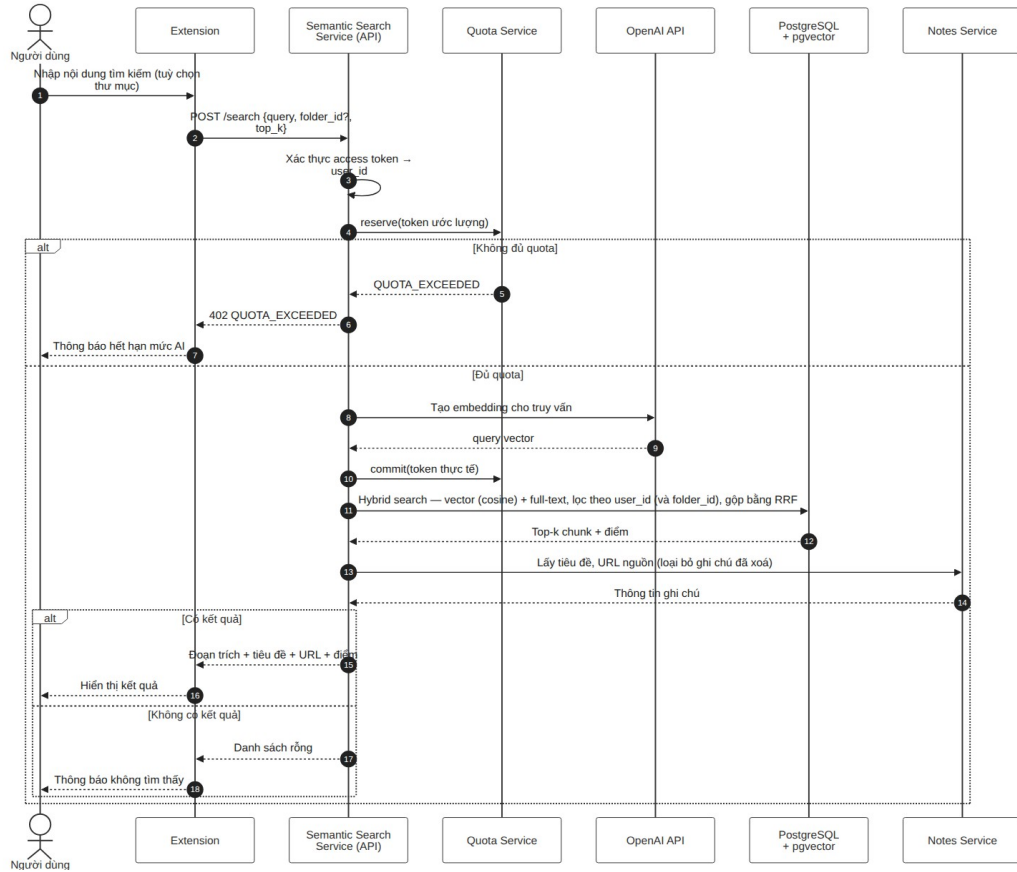
5.1.2. Sequence Diagram lưu ghi chú và tạo embedding



Hình 5. Sequence Diagram lưu ghi chú và tạo embedding

Extension ghi yêu cầu vào outbox (IndexedDB) ngay khi người dùng thao tác và cho phép hoàn tác trong 5 giây; mỗi yêu cầu mang một idempotency key nên gửi lại không tạo ghi chú trùng. Notes Service lưu ghi chú với trạng thái PROCESSING và ghi tác vụ đánh chỉ mục trong cùng một transaction, trả về ngay (201). Worker lấy tác vụ, giữ chỗ quota qua Quota Service; nếu hết quota, ghi chú chuyển SKIPPED và không bị trừ quota. Nếu đủ quota, Worker chia đoạn, gọi OpenAI tạo vector, lưu chunk và vector, chuyển trạng thái READY và chốt số token thực tế. Lỗi tạm thời được thử lại với exponential backoff; lỗi vĩnh viễn hoặc hết lượt thử chuyển FAILED để người dùng có thể “Thử lại”. Extension cập nhật trạng thái qua đồng bộ tăng dần.

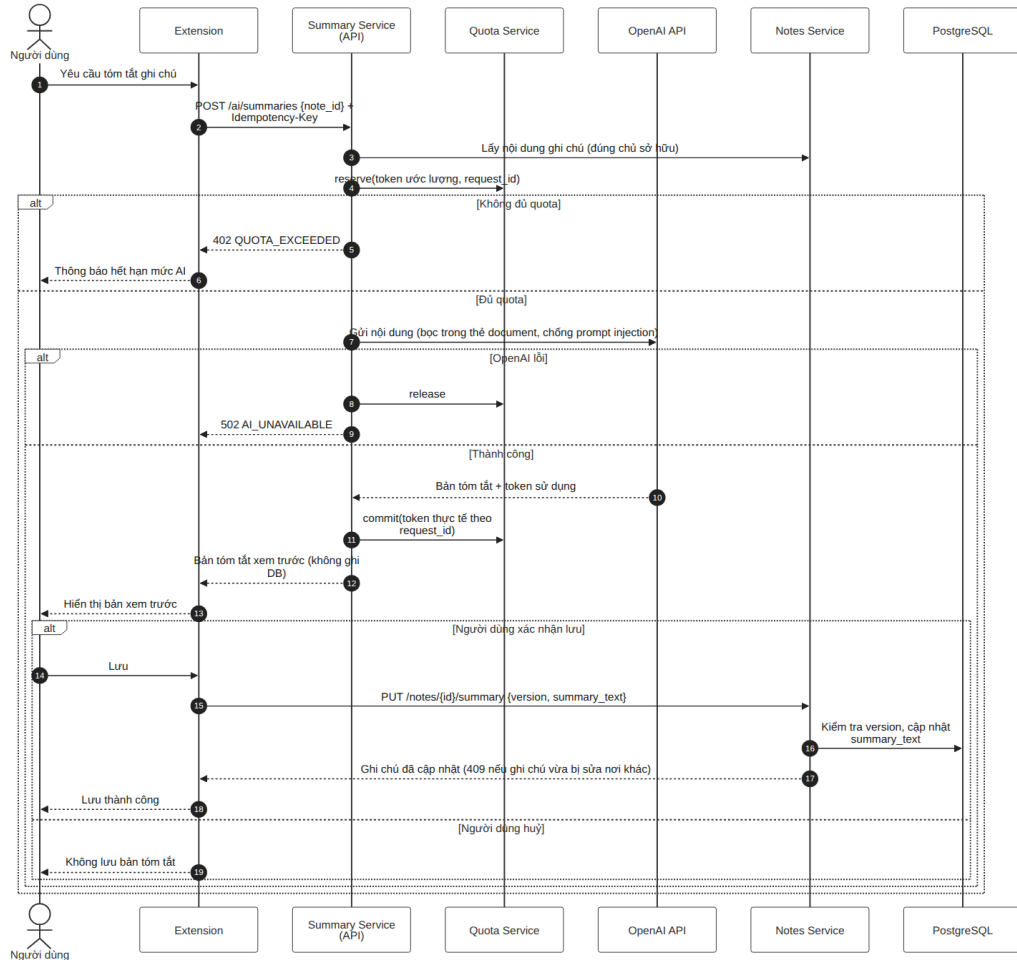
5.1.3. Sequence Diagram tìm kiếm ngữ nghĩa



Hình 6. Sequence Diagram tìm kiếm ngữ nghĩa

Truy vấn được chuyển thành vector qua OpenAI API sau khi kiểm tra quota. Hệ thống tìm kiếm kết hợp hai nhánh – độ tương đồng vector (cosine, chỉ mục HNSW) và tìm kiếm từ khóa (full-text) – trong không gian dữ liệu của người dùng hiện tại (và thư mục nếu có chọn), gộp thứ hạng bằng Reciprocal Rank Fusion và trả về top-k (mặc định 5) đoạn kèm tiêu đề và đường dẫn nguồn. Ghi chú đã xóa tự động bị loại khỏi kết quả.

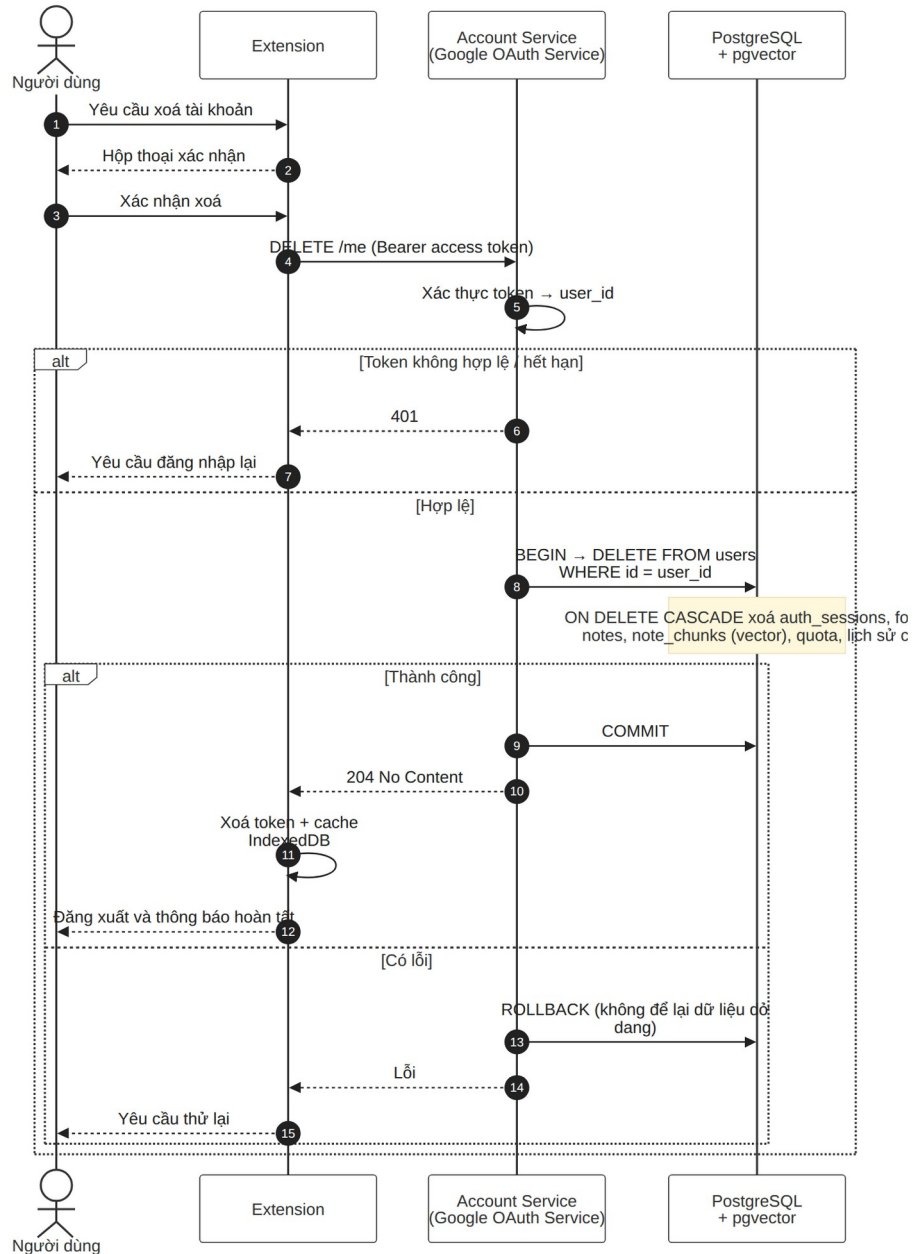
5.1.4. Sequence Diagram tóm tắt nội dung bằng AI



Hình 7. Sequence Diagram tóm tắt nội dung bằng AI

Summary Service lấy nội dung ghi chú (chỉ của chính người dùng), kiểm tra độ dài và giữ chỗ quota trước khi gọi OpenAI. Nội dung được bọc trong thẻ phân định và mô hình được dặn không làm theo chỉ dẫn bên trong văn bản (chống prompt injection). Lỗi AI hoàn lại quota; thành công thì chốt token theo request_id. Kết quả trả về dưới dạng bản xem trước; nội dung tóm tắt chỉ được ghi vào summary_text khi người dùng xác nhận, thông qua Notes Service kèm kiểm tra version.

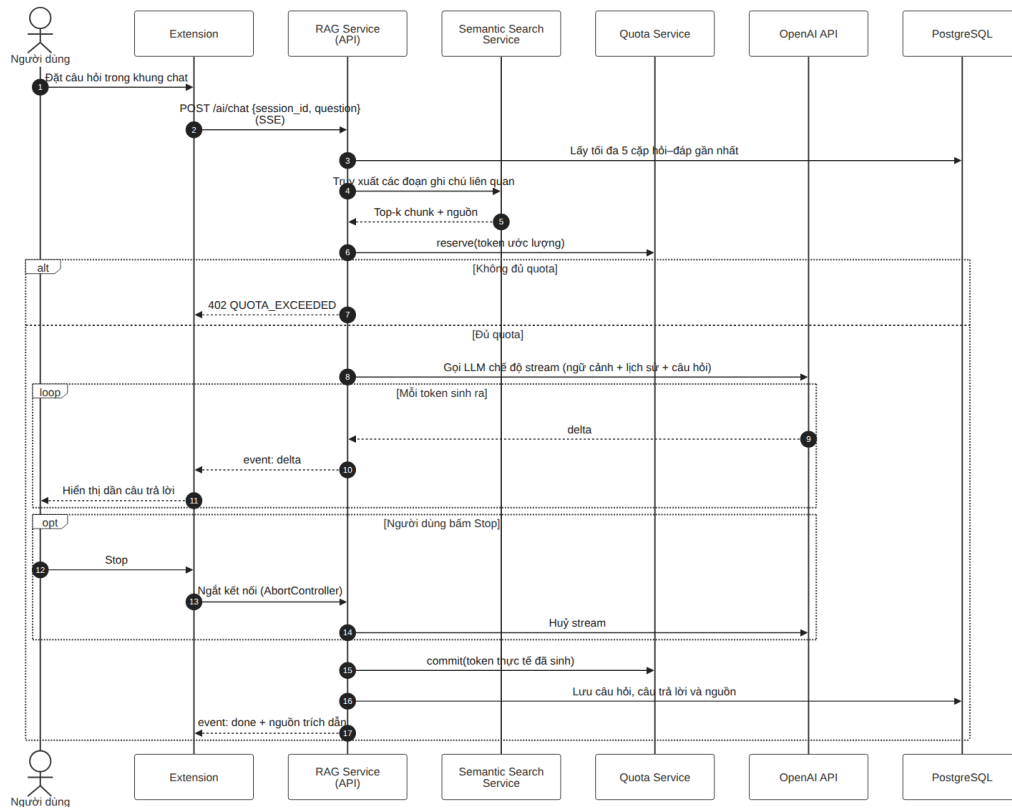
5.1.5. Sequence Diagram xóa tài khoản và dữ liệu



Hình 8. Sequence Diagram xóa tài khoản và dữ liệu

Yêu cầu xóa đòi hỏi access token hợp lệ. Account Service (thuộc Google OAuth Service, sở hữu bảng users) xóa bản ghi người dùng trong một transaction; ràng buộc khóa ngoại ON DELETE CASCADE xóa toàn bộ dữ liệu liên quan: phiên đăng nhập, thư mục, ghi chú, chunk và vector, quota, lịch sử chat. Nếu có lỗi, toàn bộ thao tác được rollback để không để lại dữ liệu dở dang. Extension xóa token và bộ nhớ đệm cục bộ sau khi xóa thành công.

5.1.6. Sequence Diagram hỏi đáp AI dựa trên ghi chú cá nhân



Hình 9. Sequence Diagram hỏi đáp AI dựa trên ghi chú cá nhân (Nice-to-have)

RAG Service lấy tối đa 5 cặp hỏi–đáp gần nhất và gọi Semantic Search Service để lấy ngữ cảnh từ các ghi chú liên quan. Khi gọi mô hình ngôn ngữ, kết quả được truyền trực tiếp (Server-Sent Events) từng phần về giao diện. Người dùng có thể nhấn nút Stop để ngắt kết nối ngay, giúp tiết kiệm quota. Việc chốt token và lưu lịch sử kèm nguồn diễn ra khi luồng kết thúc hoặc bị ngắt.

5.2. Các dịch vụ

Business Layer gồm các service dưới đây. Chúng chạy trong cùng tiến trình (API hoặc Worker), mỗi service sở hữu một nhóm bảng và chỉ cung cấp dữ liệu cho service khác qua giao diện công khai.

Bảng 4. Các service trong Business Layer

Service	Trách nhiệm	Bảng sở hữu
OAuth Service (AuthService, AccountService)	Luồng đăng nhập PKCE, định danh qua google_sub, vòng đời app session (phát, làm mới xoay vòng, thu hồi), hồ sơ và xóa tài khoản.	users, auth_sessions
Notes Service (NotesService, FolderService)	CRUD ghi chú và thư mục (Inbox mặc định), idempotency key, kiểm soát version, trạng thái xử lý AI, phát tác vụ đánh chỉ mục, lưu tóm tắt đã xác nhận.	folders, notes, idempotency_keys
Quota Service	Giữ chỗ hạn mức trước mỗi lệnh gọi OpenAI, chốt token thực tế hoặc hoàn lại theo	quota_accounts, quota_ledger

Service	Trách nhiệm	Bảng sở hữu
	request_id, báo cáo hạn mức còn lại.	
Semantic Search Service	Chia đoạn, tạo và lưu embedding (tác vụ nền), tạo vector truy vấn, tìm kiếm kết hợp trong pgvector.	note_chunks
Summary Service	Tạo bản tóm tắt xem trước, chống prompt injection, kiểm soát độ dài và quota.	—
RAG Service (Nice-to-have)	Ghép ngữ cảnh cho chatbot, quản lý lịch sử hội thoại, điều phối streaming và dừng.	chat_sessions, chat_messages (dự kiến)

5.3. Danh mục API

API có tiền tố phiên bản /api/v1, dữ liệu JSON, xác thực bằng header Authorization: Bearer <access token>. Mọi lỗi trả về dạng {"error": {"code", "message", "details", "request_id"}}; tài nguyên không thuộc người dùng trả 404 để không lộ sự tồn tại.

Bảng 5. Danh mục API chính

Phương thức và đường dẫn	Service	Ghi chú
POST /auth/google/exchange	OAuth	Đổi code + code_verifier lấy app session
POST /auth/refresh · POST /auth/logout	OAuth	Làm mới xoay vòng / thu hồi phiên
GET /me · DELETE /me	OAuth (Account)	Hồ sơ / xóa tài khoản và toàn bộ dữ liệu
GET /me/quota	Quota	Hạn mức còn lại
GET, POST /folders · PATCH, DELETE /folders/{id}	Notes	Xóa thư mục chuyển ghi chú về Inbox
POST /notes (Idempotency-Key)	Notes	201, index_status = PROCESSING
GET /notes?folder_id&updated_since&cursor	Notes	Phân trang, đồng bộ tăng dần (kèm bản ghi đã xóa)
GET, PATCH, DELETE /notes/{id} · POST /notes/bulk-delete	Notes	PATCH kèm version, lịch trả 409
PUT /notes/{id}/summary · POST /notes/{id}/reindex	Notes	Lưu tóm tắt đã xác nhận / xử lý AI lại
POST /search	Semantic Search	Tìm kiếm ngữ nghĩa, lọc thư mục
POST /ai/summaries	Summary	Trả bản xem trước, không ghi CSDL
POST /ai/chat	RAG	SSE (Nice-to-have)

6. Thiết kế dữ liệu

Hệ thống lưu trữ dữ liệu quan hệ, vector và hàng đợi tác vụ bằng PostgreSQL có cài pgvector. Khóa chính dùng UUID; thời gian lưu theo múi giờ chuẩn (timestampz). Mọi bảng chứa dữ liệu người dùng tham chiếu users.id với ON DELETE CASCADE. Lược đồ được quản lý bằng migration có phiên bản.

Bảng 6. Thiết kế dữ liệu

Thực thể	Trường chính	Ràng buộc và ghi chú
users	id, google_sub, email, display_name, avatar_url, created_at, last_active_at	google_sub duy nhất; không lưu mật khẩu; last_active_at phục vụ chính sách lưu trữ một năm
auth_sessions	id, user_id, refresh_token_hash, expires_at, revoked_at, replaced_by, user_agent	Chỉ lưu SHA-256 của refresh token; xoay vòng mỗi lần làm mới
folders	id, user_id, name, is_default, created_at, updated_at	(user_id, name) duy nhất; thư mục mặc định Inbox
notes	id, user_id, folder_id, url, domain, title, content, summary_text, index_status, index_error, version, index_rev, created_at, updated_at, deleted_at	index_status ∈ {PROCESSING, READY, SKIPPED, FAILED}; version cho kiểm soát xung đột; deleted_at (xóa mềm) phục vụ đồng bộ
note_chunks	id, note_id, user_id, folder_id, ordinal, text, content_hash, embedding vector(1536), tsv, model	Chỉ mục HNSW (cosine) trên embedding, GIN trên tsv; folder_id là bản sao để lọc theo thư mục
quota_accounts	user_id, limit_tokens, used_tokens, reserved_tokens, period_start	CHECK không âm; giữ chỗ bằng một câu UPDATE có điều kiện
quota_ledger	id, user_id, request_id, operation, reserved_tokens, actual_tokens, status, settled_at	request_id duy nhất – chống trùng; status ∈ {RESERVED, COMMITTED, RELEASED}
idempotency_keys	user_id, key, scope, request_hash, resource_id, created_at	Khóa (user_id, key); hiệu lực 24 giờ
jobs	id, kind, payload, status, attempts, max_attempts, run_after, locked_at, last_error, dedupe_key	Hàng đợi tác vụ; status ∈ {QUEUED, RUNNING, DONE, DEAD}
tags, note_tags (Nice-to-have)	id, user_id, name / note_id, tag_id	Gắn thẻ và lọc ghi chú (quan hệ N–N)
chat_sessions, chat_messages (Nice-to-have)	id, user_id, ... / session_id, role, content, sources	Lịch sử hội thoại RAG, tối đa 5 cặp gần nhất làm ngữ cảnh

- **Chỉ mục chính:** notes(user_id, updated_at, id), notes(user_id, folder_id), note_chunks(user_id, folder_id), HNSW trên embedding, GIN trên tsv, jobs(status, run_after).
- **Quyền sở hữu:** mọi truy vấn ghi chú/thư mục/chunk đều có điều kiện user_id; user_id lấy từ access token, không lấy từ dữ liệu client gửi lên.
- **Tính toàn vẹn:** ghi chú và tác vụ đánh chỉ mục được ghi trong cùng transaction; chunk cũ chỉ được thay khi chunk mới tạo thành công; index_rev đảm bảo tác vụ cũ không ghi đè trạng thái của lần sửa mới hơn.

7. Công nghệ sử dụng

Bảng 7. Công nghệ sử dụng

Thành phần	Công nghệ
Tiện ích trình duyệt	Chrome Extension Manifest V3, TypeScript, React, Vite + CRXJS; IndexedDB (idb)
Backend	Python 3.11, FastAPI, SQLAlchemy 2 (async) + asyncpg, Pydantic, Alembic
Giao tiếp	REST API /api/v1, JSON, HTTPS; Server-Sent Events cho streaming (Nice-to-have)
Xác thực	Google OAuth 2.0 + OpenID Connect (Authorization Code + PKCE); JWT app session (PyJWT)
Dịch vụ AI	OpenAI API: text-embedding-3-small (1536 chiều) cho embedding; gpt-4o-mini cho tóm tắt/RAG; Mock provider cho kiểm thử
Cơ sở dữ liệu và vector	PostgreSQL 16 + pgvector (HNSW, cosine); full-text search của PostgreSQL
Tác vụ nền	Worker Python, hàng đợi trên PostgreSQL (FOR UPDATE SKIP LOCKED)
Triển khai	Docker, Docker Compose, reverse proxy HTTPS
Chất lượng và quản lý mã nguồn	Git/GitHub, GitHub Actions (lint, test, kiểm tra kiến trúc, build extension), pytest, Vitest, Ruff

8. Bảo mật

8.1. Biện pháp bảo mật

- **Tài khoản người dùng:** Đăng nhập qua Google OAuth với PKCE, kiểm tra state (chống CSRF) và nonce; xác minh id_token (chữ ký, iss, aud, exp, email_verified); không lưu mật khẩu Google.
- **Phiên làm việc:** Access token ngắn hạn (15 phút); refresh token xoay vòng, chỉ lưu giá trị băm; phát hiện tái sử dụng refresh token thì thu hồi mọi phiên của người dùng.

- **Thông tin bí mật:** client_secret, API key và thông tin kết nối cơ sở dữ liệu chỉ nằm ở biến môi trường phía máy chủ; extension không chứa secret.
- **Truyền dữ liệu:** Mọi kết nối giữa Extension, Backend và dịch vụ ngoài sử dụng HTTPS; CORS chỉ cho phép origin của extension.
- **Quyền truy cập:** user_id lấy từ access token; mọi truy vấn kiểm tra quyền sở hữu; tài nguyên không thuộc người dùng trả 404 (chống IDOR).
- **Quyền Extension:** Quyền tối thiểu – không có content script chạy trên mọi trang, không dùng <all_urls>; chỉ đọc vùng chọn trên tab người dùng vừa thao tác (activeTab); chỉ thu thập nội dung khi người dùng chủ động yêu cầu.
- **Nội dung trang web:** Xử lý như dữ liệu không tin cậy: giới hạn độ dài, làm sạch trước khi lưu, không hiển thị bằng innerHTML; bọc nội dung khi gửi LLM để chống prompt injection.
- **Log:** Log dạng JSON có request_id; tự động che token, API key và nội dung ghi chú.
- **Xóa dữ liệu:** Xóa tài khoản xóa toàn bộ dữ liệu liên quan trong một transaction và xóa bộ nhớ đệm trên máy người dùng.

9. Triển khai

- **Khả năng mở rộng:** API không lưu trạng thái phiên nên chạy nhiều instance phía sau reverse proxy/Load Balancer; Worker chạy song song an toàn nhờ SKIP LOCKED; giới hạn thực tế là kết nối CSDL và quota OpenAI (§4.5).
- **Cấu hình:** OpenAI API key, Google OAuth credential, kết nối PostgreSQL, JWT secret, quota, mô hình AI, kích thước chunk, top-k và timeout đặt trong biến môi trường (tệp .env không đưa vào Git).
- **Phụ thuộc:** Chrome/Chromium hỗ trợ Manifest V3, Python 3.11, PostgreSQL 16 có pgvector, Google OAuth và OpenAI API.
- **Môi trường triển khai:** Giai đoạn phát triển chạy Docker Compose trên máy cá nhân với AI Mock; môi trường demo chạy trên cloud/VPS với OpenAI thật. Thứ tự khởi động: PostgreSQL → migration → API → Worker.
- **Self-host:** Chỉ là định hướng phát triển sau MVP.

Kiểm tra trước khi demo:

- 1. Khởi chạy PostgreSQL + pgvector, chạy migration, khởi chạy API và Worker; kiểm tra /api/v1/health/ready.
- 2. Cấu hình Google OAuth (redirect URI theo Extension ID) và OpenAI API.
- 3. Cài Extension trên Chrome.
- 4. Kiểm tra đăng nhập và đăng xuất.
- 5. Kiểm tra lưu ghi chú bằng menu chuột phải, phím tắt và hoàn tác.
- 6. Kiểm tra quản lý thư mục, thẻ và ghi chú.
- 7. Kiểm tra tóm tắt, embedding và tìm kiếm ngữ nghĩa.
- 8. Kiểm tra quota và các trạng thái lỗi (hết quota, mất mạng, lỗi AI).
- 9. Kiểm tra xóa tài khoản.

- 10. Kiểm tra RAG nếu chức năng này được triển khai.

10. Xử lý lỗi và Giảm thiểu rủi ro

Bảng 8. Rủi ro và biện pháp giảm thiểu

Rủi ro / lỗi	Biện pháp giảm thiểu
Google OAuth thất bại hoặc phiên hết hạn	Kiểm tra token, tự làm mới bằng refresh token (một lần, tránh gọi song song); thất bại thì yêu cầu đăng nhập lại.
Ghi chú bị lưu trùng	Idempotency key cho mỗi thao tác lưu; outbox phía extension gửi lại với cùng key.
Mất mạng khi lưu ghi chú	Ghi chú nằm trong outbox (IndexedDB), tự gửi lại theo lịch; không mất khi service worker bị tắt.
OpenAI API lỗi hoặc timeout	Timeout, retry có giới hạn với exponential backoff; hoàn lại quota; ghi chú FAILED có thể thử lại.
Không đủ hoặc trừ sai quota	Giữ chỗ trước khi gọi AI; chốt theo request_id duy nhất để chống trừ trùng.
AI tóm tắt hoặc tìm kiếm chưa chính xác	Tìm kiếm kết hợp vector + từ khóa; bộ đánh giá Recall@k; người dùng xác nhận bản tóm tắt.
Cơ sở dữ liệu hoặc xử lý nền gặp lỗi	Lưu ghi chú gốc trước, tác vụ ghi cùng transaction; worker thử lại, tác vụ hỏng chuyển DEAD để điều tra.
Truy cập trái phép hoặc rò rỉ dữ liệu	Kiểm tra quyền sở hữu ở mọi truy vấn, trả 404; kiểm thử IDOR; không ghi log dữ liệu nhạy cảm.
Mã nguồn lệch kiến trúc khi nhiều người cùng phát triển	Kiểm tra kiến trúc tự động trong CI (luật phụ thuộc tầng); CODEOWNERS theo thư mục.
Extension không tương thích	Sử dụng API chính thức của Manifest V3 và kiểm thử trên các trình duyệt Chromium mục tiêu.